



Lab Course Automatic Control II
Prof. Dr.-Ing. Knut Graichen
Chair of Automatic Control



Lab Course Automatic Control II

Flatness-based control of a ball-on-plate system

Supervisor: Daniel Landgraf

1 Introduction

In this practical experiment, a *ball-on-plate* system is considered. The moving ball on the plate represents a nonlinear dynamic system. The setup, which is shown in Figure 1, consists of an inner and an outer frame, whose rotation axes are orthogonal to each other. This allows to move the ball to any point on the plate by tilting the frames suitably. The rotation of the plates is generated by the control with two DC motors, which transmit the torque to the frames using a toothed belt drive. In order to determine the current ball position, a touch screen is used on the plate surface. It consists of two conductive layers and an insulating separator. As soon as the ball is on the plate, the conductive layers are in contact with each other and the position can be determined by applying reference voltages. Two further sensors are located directly on the rotation axes of the two frames to determine the angles.

In this experiment, a controller is designed with the objective of keeping the ball on a given trajectory. This task is implemented using *flatness-based trajectory tracking control*. For a trajectory tracking control, the desired trajectory, which explicitly depends on time, contains both the geometric trajectory and, via the time parameterization, the dynamics of the ball, whereby a specific position *and* velocity of the ball is expected at any time. Thus, a trajectory tracking control corresponds to following a time-varying target position. The designed nonlinear controller is first validated by simulation and then tested on the experimental setup, for which a real-time dSPACE system is available.

For the execution of this experiment the contents of the lecture *Nonlinear Control Systems* are presupposed. If necessary, read through the most important topics again with the help of [1].

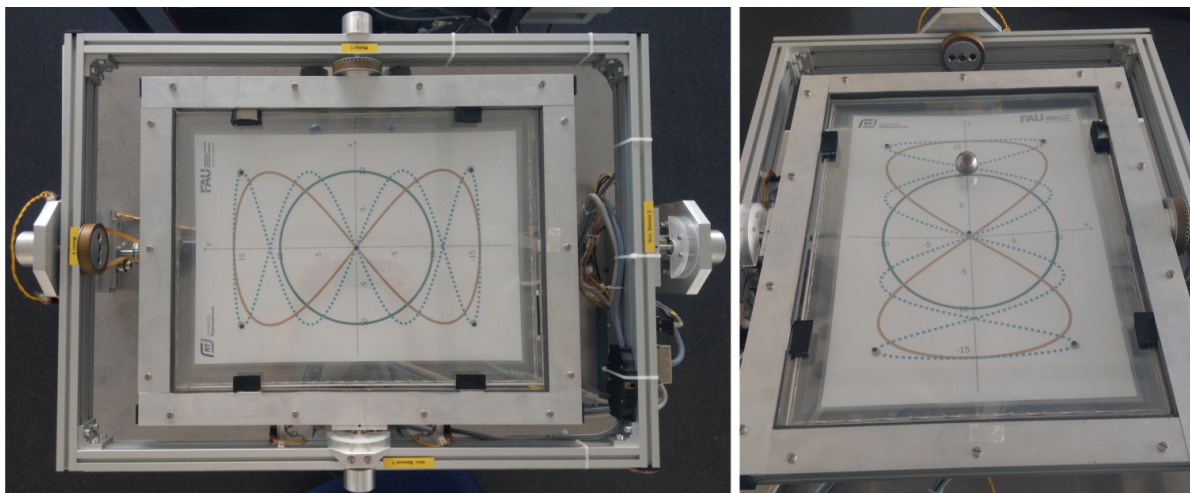


Figure 1: The ball-on-plate system.

2 Preparation

The basis for the controller design is a system model, which is derived in a first step. For this purpose, the system is decomposed into two separate *ball-on-stick* systems, which are subsequently considered independently of each other. The geometric relations of a ball-on-stick system are shown in Figure 2. In the modeling, it is assumed that the ball is always in contact with the rod and moves only by rolling around its center of gravity, i.e., the ball neither bounces nor slides. Choosing the angle θ and the ball position x_B as generalized coordinates of the system, the equations of motion can be calculated using the *Euler-Lagrange equations* with

$$0 = \left(\frac{J_B}{r_B^2} + m_B \right) \ddot{x}_B + m_B g \sin(\theta) - m_B x_B \dot{\theta}^2 \quad (1)$$

$$\tau = (m_B x_B^2 + J_S + J_B) \ddot{\theta} + 2m_B x_B \dot{x}_B \dot{\theta} + m_B g x_B \cos(\theta). \quad (2)$$

Here J_B denotes the moment of inertia of the ball, m_B denotes its mass, r_B denotes its radius, J_S denotes the moment of inertia of the rod, g denotes the gravitational acceleration and τ denotes the torque acting on the rod. If the angular acceleration $\ddot{\theta}$ is now chosen as the new input u , by defining

$$B = \frac{m_B}{\frac{J_B}{r_B^2} + m_B} \quad (3)$$

and using the nonlinear invertible transformation

$$\tau = 2m_B x_B \dot{x}_B \dot{\theta} + m_B g x_B \cos(\theta) + (m_B x_B^2 + J_S + J_B) u, \quad (4)$$

the system can be transformed into the state space representation

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} x_2 \\ B(x_1 x_4^2 - g \sin(x_3)) \\ x_4 \\ u \end{bmatrix} \quad (5)$$

with the state vector $\mathbf{x} = [x_B, \dot{x}_B, \theta, \dot{\theta}]^T$, see [2].

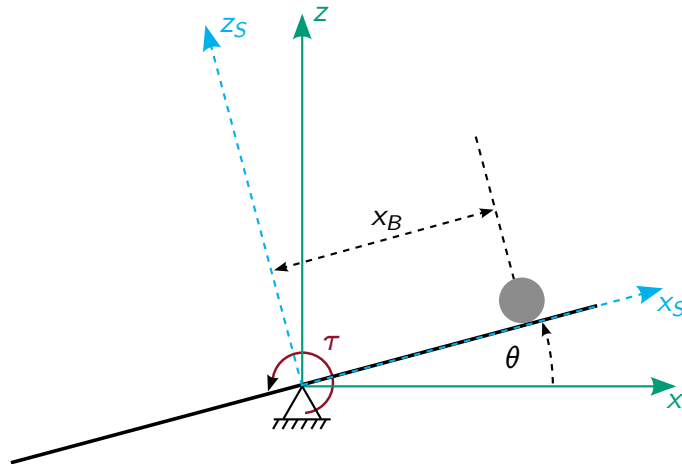


Figure 2: Geometric variables of a ball-on-stick system.

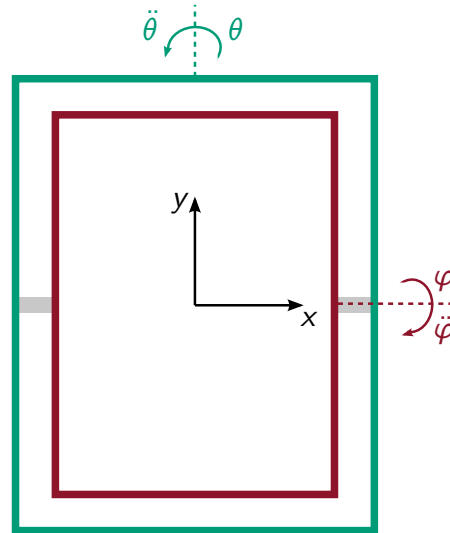


Figure 3: Definition of the coordinates.

Since the system is not flat in this form, the simplified system

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} x_2 \\ -Bg \sin(x_3) \\ x_4 \\ u \end{bmatrix} \quad (6)$$

is considered.

(P1) Show that $z = x_B$ is a flat output of the system and calculate the flatness parameterization $\Psi(\cdot)$ for the states $\mathbf{x}$ and for the control input u . Is the system statically feedback linearizable?

The model of the complete ball-on-plate system is then obtained by interconnecting two independent ball-on-stick systems. The coordinate definition used in this process is shown in Figure 3. The outer frame can be rotated around the y -axis, where a positive rotation angle θ leads to an acceleration of the ball in the negative x -direction. Similarly, the inner plate, which is rotatably mounted in the outer plate, can be rotated around the x -axis. Thus, with the state and control input vectors

$$\mathbf{x} = [x_B, \dot{x}_B, \theta, \dot{\theta}, y_B, \dot{y}_B, \varphi, \dot{\varphi}]^T \quad (7)$$

$$\mathbf{u} = [\ddot{\theta}, \ddot{\varphi}]^T, \quad (8)$$

the complete system dynamics results in

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \\ \dot{x}_5 \\ \dot{x}_6 \\ \dot{x}_7 \\ \dot{x}_8 \end{bmatrix} = \begin{bmatrix} x_2 \\ -Bg \sin(x_3) \\ x_4 \\ u_1 \\ x_6 \\ -Bg \sin(x_7) \\ x_8 \\ u_2 \end{bmatrix}. \quad (9)$$

The ball has a mass of $m_B = 0.092$ kg, a radius $r_B = 14.3$ mm, and its moment of inertia can be calculated by $J_B = \frac{2}{5} m_B r_B^2$.

- (P2) Calculate the flatness parameterization $\Psi(\cdot)$ of the states $\mathbf{x}$ and of the control input $\mathbf{u}$ for the complete system (9) for the flat output $\mathbf{z} = [x_B, y_B]^T$.

In a first step, a setpoint change of the ball from a starting position $\mathbf{z}_0^*$ to a target position $\mathbf{z}_T^*$ is realized for the system. To this end, n -times continuously differentiable *polynomial desired trajectories* of the form

$$\mathbf{z}^*(t) = \mathbf{z}_0^* + (\mathbf{z}_T^* - \mathbf{z}_0^*) \sum_{i=n+1}^{2n+1} p_i \left(\frac{t}{T}\right)^i, \quad t \in [0, T] \quad (10)$$

are considered, where T denotes the *transition time*.

- (P3) Determine a desired trajectory for the setpoint change of the x - and y -position of the ball. Then specify the optimal controls $u_1^*(t)$ and $u_2^*(t)$ and determine the derivatives of the polynomial desired trajectories that are necessary for the calculation.
- (P4) Implement your feedforward control using Simulink and simulate the ball-on-plate system for different setpoint changes. Use the simplified dynamics (9), which was also used for the design of the feedforward control. Start once exactly on the nominal trajectory and once with an initial deviation (i.e. $\mathbf{z}_0 \neq \mathbf{z}_0^*$). What principal behavior do you observe?

Hint: Compute the required derivatives of the polynomial trajectories analytically rather than numerically.

The actual objective of the experiment is to move the ball along a periodic trajectory. The periodic desired trajectories considered in the experiment, a circle, an eight and a double eight, can be calculated explicitly with the help of sine and cosine by

$$\mathbf{z}^*(t) = \begin{bmatrix} x_B^*(t) \\ y_B^*(t) \end{bmatrix} \quad \text{with} \quad x_B^*(t) = a_x \sin(b_x v t), \quad y_B^*(t) = a_y \cos(v t). \quad (11)$$

The parameters a_x and a_y indicate the amplitude in the respective direction, the parameter v determines the velocity and by the parameter b_x the type of the target trajectory can be selected (circle $b_x = 1$, eight $b_x = 2$, double eight $b_x = 4$).

- (P5) Simulate the system with your feedforward control in Simulink for the periodic desired trajectories (11). Choose the circle as target trajectory ($b_x = 1$) with the parameters $a_x = a_y = 0.1 \text{ m}$ and $v = 1.0 \text{ rad s}^{-1}$. Start once exactly on the nominal trajectory and once with an initial deviation (i.e. $\mathbf{z}(0) \neq \mathbf{z}^*(0)$). What principal behavior do you observe?

Hint: Since there are no stationary boundary values for these periodic nominal trajectories, you need to initialize the initial state $\mathbf{x}_0$ of your system (except for the starting position of the ball) depending on the nominal trajectories (11) and their derivatives at time $t = 0$. You can calculate the values using the flatness parameterization Ψ_x . Again, compute the required derivatives of the target trajectories analytically rather than numerically.

What happens if you choose $\mathbf{x}_0$ not dependent on the desired trajectories?

Now we will investigate how model uncertainties affect the result when using feedforward control only.

(P6) *For this purpose, replace the plant model in your Simulink model with the non-simplified dynamics (5). Then test the feedforward control for the eight as the desired trajectory with the parameters*

$$a_x = 0.1 \text{ m}, \quad a_y = 0.16 \text{ m}, \quad v = 1.0 \text{ rad/s}, \quad b_x = 2 \quad (12)$$

and a simulation time of 100 s. What behavior do you observe during the simulation.

To compensate for initial deviations, disturbances, and model inaccuracies, a flatness-based tracking controller will be designed.

(P7) *Design a flatness-based tracking controller for the ball-on-plate system. Choose the Hurwitz coefficients such that all eigenvalues of the closed loop system are at -4 .*

Note: The aim of the experiment is to implement and test your controller developed in the preparation on the real system. Therefore, bring all relevant files (Simulink model, initialization script, etc.) from the experiment preparation to the lab session. Since MATLAB R2018b is installed on the lab computer, save your Simulink model in this version if you are using a newer MATLAB version to avoid complications (with File → Export Model to → Previous Model...).

3 Experiments

During the execution of the experiment, the flatness-based tracking controller designed in the preparation is implemented, simulatively validated and then tested on the real system. Before you start with the implementation, carefully read the general notes on carrying out experiments in Section 3.1.

3.1 General instructions

- Make sure at all times that you keep sufficient distance to the moving parts of the experiment during execution, as large and erratic movements in the plate angles can occur in the event of errors in implementation / operation!
- If one of the frames reaches one of the end stops during the experiment, the system will automatically switch off. In this case, deactivate the control, remove the ball from the plate and manually return the system to the horizontal position. Then press the reset button in the front of the computer case.
- Save at least one measurement for each task of the experiment execution. This is done by right clicking in the plot window in ControlDesk and then clicking "Save Displayed Data as New Measurement".
- Pay particular attention to the instructions in the individual tasks when carrying them out.

3.2 Practical execution

Open MATLAB at the computer. If a query about the dSPACE system appears, select "RTI1202". Acknowledge all further messages with OK. Then change to the directory Documents\ball-OnPlate_flatness\code and open the model ballOnPlate_flatness_simulation.slx, in which the non-simplified model (5) is implemented as well as the corresponding initialization script Initialization_Simulation.m. After a simulation run, the motion of the ball is animated and the relevant variables are displayed.

- (E1) *Implement your flatness-based tracking controller designed in the experiment preparation into the Simulink model. The current system states $\mathbf{x}_k$ and the current time t_k are available as inputs. In addition, add your reference trajectory calculation used in the preparation for both the setpoint change and the periodic reference trajectories and link the outputs with the calculated control inputs as well as the current position of the ball. Implement a mechanism to switch between the two different working modes depending on a parameter (e.g. via the Switch block).*
- (E2) *Test the same use cases as in the preparation tasks (P4) – (P6). What are the differences compared to using feedforward control only?*

After you have successfully validated the tracking controller by simulation, open the Simulink model ballOnPlate_flatness.slx. In this model, signal processing and actuator control have already been implemented.

- (E3) *Integrate your implemented flatness-based controller including the calculation of the target trajectories into the existing Simulink template in Control → Flatness-based control. Available inputs are the current measured system states $\mathbf{x}_k$ and the current time t_k .*

In addition, transform the outputs of your controller from angular acceleration to torque using (4). To do so, choose the moments of inertia for the inner and outer plates to be $J_{S,i} = 0.111\,08\text{ kg m}^2$ and $J_{S,a} = 0.346\,02\text{ kg m}^2$.

Link the outputs with the calculated control inputs and the current target position of the ball.

To be able to use the designed controller on the real system, the Simulink model must be compiled and loaded to the dSPACE system. This can be done either by the *build code* icon in Simulink (rightmost in the top toolbar) or by the key combination *Ctrl-B*. Additionally, this will create a *variable description* file (*.sdf*) in the current directory that will be needed later. Make sure that the dSPACE Micro Lab Box is switched on, otherwise an error message will occur during compilation.

- (E4) *Execute the already existing script Initialization.m, which contains the parameters for the control of the actuators as well as the signal processing. Compile the Simulink model and fix any error messages that may occur in the process.*

In the next step, start the *dSPACE ControlDesk* program and open the prepared project via *File → Open → Open Project and Experiment → Root Directory → Symbol “new folder” → Documents\ballOnPlate_flatness\Laboratory_RT2_flatness_ballOnPlate* select → *OK*.

The project contains a graphical user interface for controlling the ball-on-plate system and for displaying the setpoint and actual position of the ball, its velocity, and the control variables. With the *Go Online* and *Start Measuring* icons in the upper toolbar you start the online control and the display of the measured values. Before compiling a new version of your Simulink model, you must click *Go Offline*, otherwise an error message will appear during the build process. After **each** compile, you must load the new version of the *.sdf* file in *ControlDesk* using *Project → Lab_flatness_ballOnPlate\Hardware Configurations\Platform\ballOnPlate_flatness.sdf → RightClick → Reload → Yes*.

To enable efficient parameterization of your flatness-based controller in online mode at the experimental setup, it makes sense to introduce the most important parameters (gain factors in the control law or the eigenvalues, parameters that determine the control mode, and the parameters of the target trajectories from (11) or the target point and the transition time for the setpoint change) as variables that can be adjusted via the *ControlDesk* interface. This enables parameterization in online mode without having to recompile the Simulink model each time.

- (E5) *Define the relevant parameters as MATLAB variables in an initialization script. Then use these parameters in the Simulink model for the setpoint calculation and in the control law.*

*To be able to customize the parameters in the ControlDesk interface, insert an input field for each variable via *Layouting → Insert Instrument → Numeric Input*. Then you can link the respective variable to the input field by *Right click on input field → Variables → Add*.*

Note: *You must first recompile your Simulink model after making the changes and reload the .sdf file, otherwise the parameters will not yet be present in ControlDesk.*

Now your developed control system is tested and parameterized on the real system.

- (E6) Switch on the motors of the ball-on-plate system (switch at the back of the PC case) and press the reset button on the front of the PC case. If one of the two plates reaches its end stop during the test, the reset button must be pressed again. Then switch to online mode in the ControlDesk interface. Enable the motors for the two plates and switch to the “angle control” mode. This aligns the plates horizontally. In the mode “approach start position”, the ball is moved to the desired start position.

Note: Always place the ball on the plate in “angle control” mode before subsequently changing to “approach start position” mode to ensure that a reasonable position measurement of the ball is available to the controller.

Bring the ball to the origin and then switch to “flatness-based control” mode and test your controller first for stabilizing the origin by adjusting the parameters accordingly. To do this, carefully deflect the ball manually from the origin. Adjust the eigenvalues to bring the ball back to the origin without excessive overshoot. Is the origin accurately stabilized at steady state?

Next, test a setpoint change from $\mathbf{z}_0^* = [0, 0]^T$ m to $\mathbf{z}_T^* = [0.1, 0.1]^T$ m with a transition time of $T = 2$ s. Does the ball follow the desired trajectory?

In the following, the developed controller for tracking the periodic desired trajectories will be studied.

- (E7) First test your controller for the circular path as the desired trajectory with the parameters

$$a_x = 0.1 \text{ m}, \quad a_y = 0.1 \text{ m}, \quad v = 1.0 \text{ rad/s}, \quad b_x = 1. \quad (13)$$

Adjust the gain factors in the control law or the eigenvalues to obtain the best possible trajectory tracking.

Note: Bring the ball to the position $\mathbf{z}^*(0) = [x_B^*(0), y_B^*(0)]^T$ in “approach start position” mode before switching to “flatness-based control” mode.

- (E8) Manually disturb the system by “pushing” the ball slightly out of its path. Can the controller compensate for these manual disturbances? What behavior do you observe when you manually stop the ball at a point?

Then test the control for the simple eight as the desired trajectory with the parameters

$$a_x = 0.1 \text{ m}, \quad a_y = 0.16 \text{ m}, \quad v = 0.5 \text{ rad/s}, \quad b_x = 2 \quad (14)$$

with the same gain factors in the control law or eigenvalues as for the circle. Does this give a good control performance for this desired trajectory as well?

- (E9) Finally, try to move the outer frame manually by deactivating the motor for this frame while the inner one continues to be moved by the controller. Can you manually achieve similar accuracy as the controller?

References

- [1] K. Graichen. Nonlinear Control Systems. *Lecture notes, Friedrich-Alexander Universität Erlangen-Nürnberg*, 2023.
- [2] J. Hauser, S. Sastry, P. Kokotovic. Nonlinear control via approximate input-output linearization: The ball and beam example. *IEEE transactions on automatic control*, 37(3), 392-398, 1992.